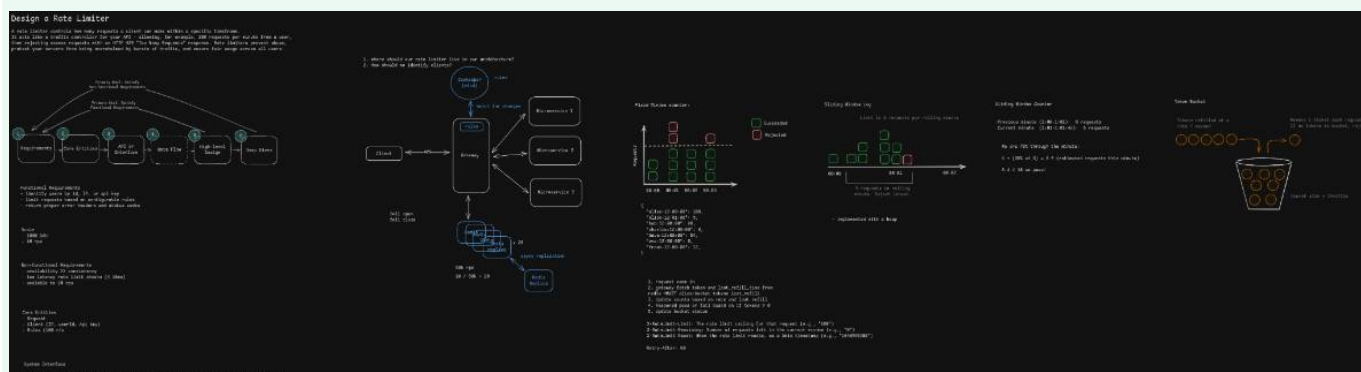


Distributed Cache HLD Design



Design an Hotel Booking System (AirBnb / Makemytrip / Bookings.com)

- Online platform that allows users to search and book rooms based on location, date and availability.

1. Functional Requirement:

- user should be able to create account in our application.
- Search for hotels based on name, location, dates
- user should be able to select a hotel room based on above preference.
- Make the payment to confirm booking
- View past and future booking details on profile

Non-Functional Requirement:

- Scale: 50M users, 1M hotels
- CAP Theorem: Availability >>> consistency. Application should be highly available based on searching. And system should be highly consistent based on booking. No two user should be allowed to book the same room/hotel on same time frame.

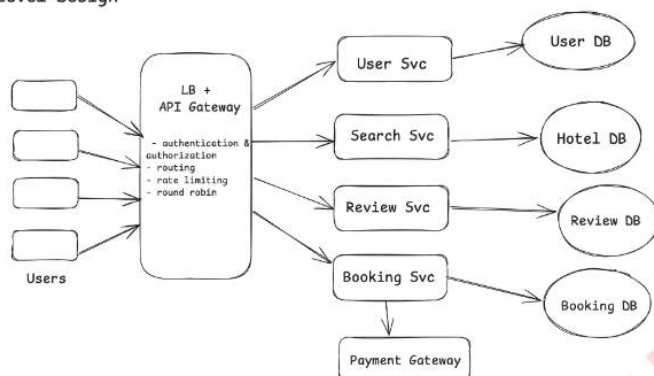
2. Identify Core Entity

- User
- Hotel
- Rooms
- Booking

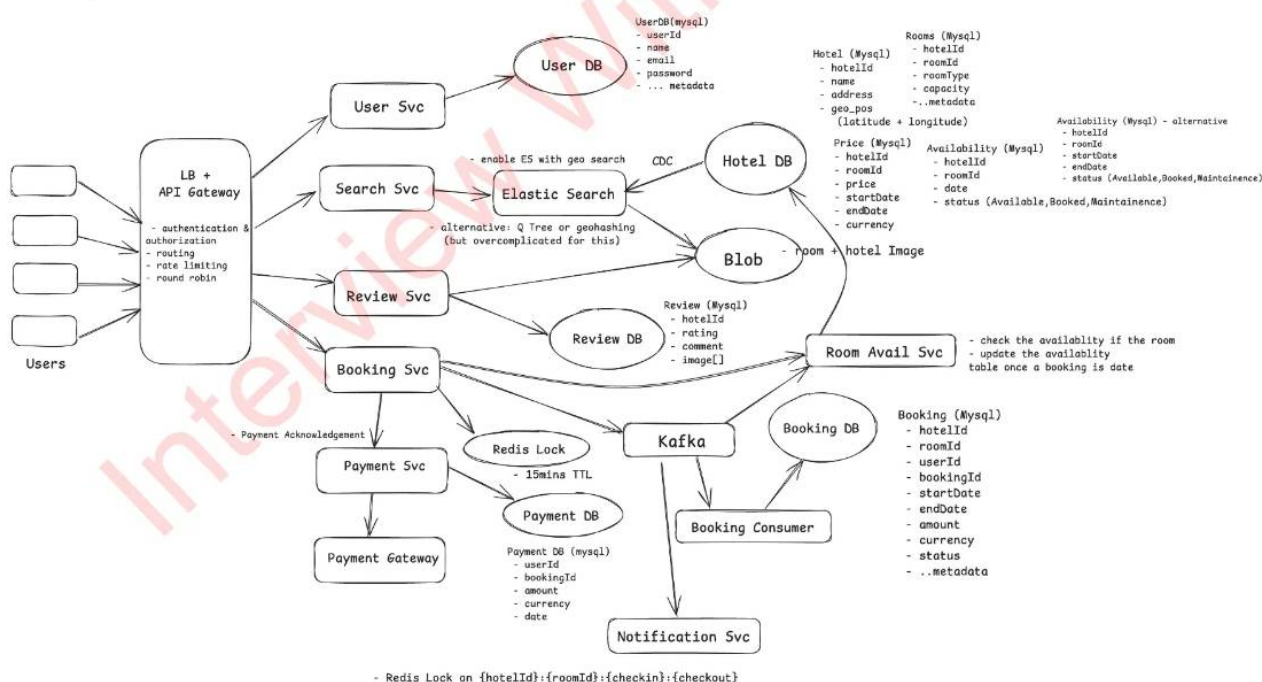
3. API Designing

1. POST /v1/user/register + (login/logout/update)
2. GET : /v1/hotels/search?q={name}&l={location}&d={dates} -> List<HotelId(partial)> : Pagination
3. GET: /v1/hotels/room/{hotelId}/{dates}
4. POST: /v1/booking/roomId {postbody: userId, dates} -> Return Booking ID
5. PUT: /v1/booking/bookingId (modify + cancel booking):
6. GET: /v1/bookings {header:userId} -> List of bookings

4. High Level Design



4. Low Level Design



can you please create a hld for a distributed cache like redis, cassandra, please consider the attached hld diagram of a hotel booking, rate limiter, you have design the hld in the given attached format.

Design a Distributed Cache System (Redis / Cassandra Style)

1. Functional Requirements

- Store key-value pairs with low latency
- Support **GET**, **SET**, **DELETE**
- Support TTL (time-to-live) expiration
- Distributed across multiple nodes
- Horizontal scalability
- High availability with replication
- Eviction policies (LRU / LFU)
- Consistent hashing for sharding
- Support persistence (snapshot + append logs)
- Pub/Sub support (optional)
- Cache invalidation support
- Atomic operations (**INCR**, **DECR**)
- Handle node failures gracefully

Non-Functional Requirements

- Scale: 10M+ QPS
- Read latency < 2ms
- Write latency < 5ms
- High availability (99.99%)
- Partition tolerant
- Eventually consistent replication
- Auto rebalancing on node addition/removal
- Memory efficient
- Support multi-region deployment

2. Identify Core Entities

- Cache Entry
- Cache Node
- Cluster Metadata
- Partition / Shard
- Replication Group
- WAL (Write Ahead Log)
- SSTable / Persistent Storage
- Client SDK
- Consistent Hash Ring

3. API Designing

```

http
SET /v1/cache
Body:
{
  "key": "user:101",
  "value": "{...}",
  "ttl": 300
}

GET /v1/cache/{key}

DELETE /v1/cache/{key}

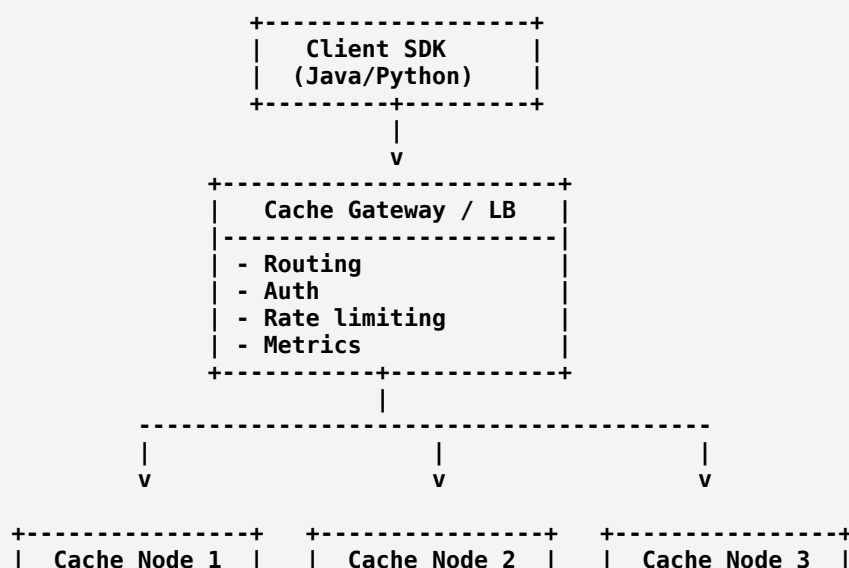
INCR /v1/cache/{key}

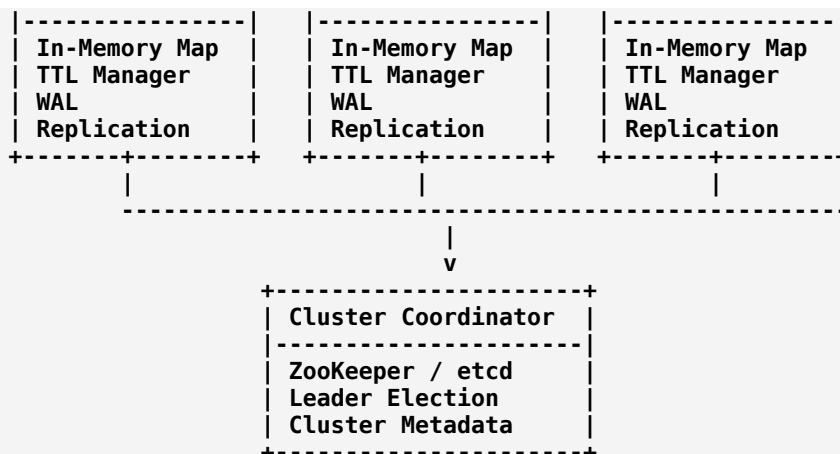
DECR /v1/cache/{key}

BULK_GET /v1/cache/bulk
Body:
{
  "keys": ["k1", "k2"]
}

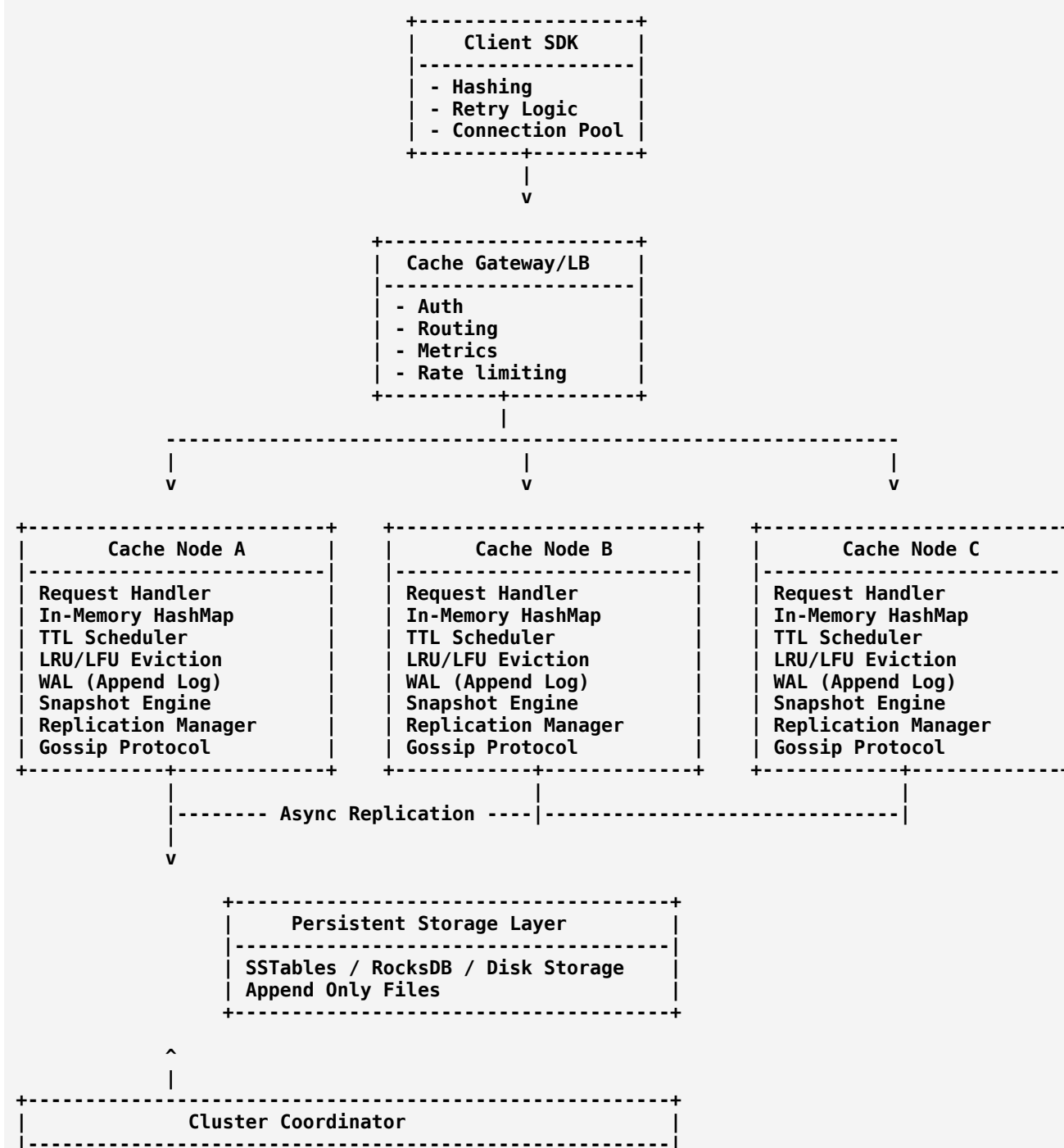
```

4. High Level Design





5. Low Level Design



```
| - Node Discovery
| - Metadata Management
| - Leader Election
| - Failure Detection
| - Rebalancing
+-----+
```

6. Data Model

Cache Entry

JSON

```
{
  "key": "user:101",
  "value": "{userData}",
  "ttl": 300,
  "createdAt": 1710000000,
  "expireAt": 1710000300,
  "version": 12
}
```

7. Internal Data Structures

In-Memory Store

Java

```
ConcurrentHashMap<String, CacheEntry>
```

Used for ultra-fast lookups.

TTL Management

Min Heap

```
(expiryTime, key)
```

Why?

- Quickly identify expired keys
- $O(\log n)$ insertion/removal
- Background cleanup thread

LRU Eviction

```
HashMap<Key, Node>
DoublyLinkedList
```

Why?

- $O(1)$ access
- $O(1)$ eviction

8. Partitioning (Sharding)

Consistent Hashing

```
hash(key) % N
```

Better approach:

```
Consistent Hash Ring
```

Benefits:

- Minimal data movement
- Easy scaling
- Node addition/removal efficient

Example:

```
user:101 -> Node B
order:901 -> Node C
cart:111 -> Node A
```

9. Replication Strategy

Primary-Replica Model

```
Primary Node
|
|---- Replica 1
|
|---- Replica 2
```

Write Flow

1. Client sends SET
2. Primary updates memory
3. WAL appended
4. Async replication to replicas
5. ACK returned

10. Read/Write Flow

SET Operation

1. Client SDK hashes key
2. Route to responsible node
3. Write to memory
4. Write to WAL
5. Replicate asynchronously
6. Return success

GET Operation

1. Route request
2. Lookup HashMap
3. Check TTL
4. Return value
5. Update LRU

11. TTL Expiration Design

Lazy Expiration

Expired key removed during read.

Pros:

- Cheap

Cons:

- Memory wastage

Active Expiration

Background scheduler removes expired keys periodically.

Used in Redis.

12. Persistence Design

Snapshotting

Take full memory snapshot every N minutes

Pros:

- Faster recovery

Cons:

- Possible data loss between snapshots
-

Append Only File (AOF)

```
SET user:1 abc  
SET user:2 xyz  
DELETE user:1
```

Pros:

- Durable

Cons:

- Large logs

Best practice:

- Use both Snapshot + AOF
-

13. Failure Handling

Node Failure

- Coordinator detects heartbeat failure
 - Promote replica as primary
 - Update hash ring
 - Rebalance partitions
-

Split Brain

Use:

- ZooKeeper
 - etcd
 - Raft consensus
-

14. Eviction Policies

LRU (Least Recently Used)

Evict least recently accessed item.

Good for:

- General workloads

LFU (Least Frequently Used)

Evict least frequently used item.

Good for:

- Stable hot keys

15. Cassandra Style Design (Wide Column Cache)

If designing like Cassandra:

Architecture

```

Client
|
Coordinator Node
|
Partitioner
|
Replica Nodes
  
```

Concepts

- Consistent hashing
- Gossip protocol
- SSTables
- Memtables
- Compaction
- Tunable consistency

Cassandra Write Path

```

Write Request
|
Commit Log
|
Memtable
  
```

16. Redis vs Cassandra

Feature	Redis	Cassandra
Storage	In-memory	Disk-first
Latency	Ultra low	Low
Persistence	Optional	Strong
Scaling	Cluster mode	Native
Query Model	Key-value	Wide column
Best Use	Cache	Huge distributed DB
Replication	Async	Multi-master
Consistency	Eventual	Tunable

17. Bottlenecks & Optimizations

Hot Keys

Problem:

Single key gets massive traffic.

Solutions:

- Local caching
- Replicated hot keys
- Request coalescing

Large Objects

Solutions:

- Compression
- Chunking
- Store blobs externally

Cache Stampede

Use:

- Mutex lock
 - Request collapsing
 - Probabilistic early expiration
-

18. Monitoring & Observability

Track:

- Cache hit ratio
- Eviction count
- Replication lag
- Memory usage
- Hot keys
- P99 latency
- QPS

Tools:

- Prometheus
 - Grafana
 - ELK
-

19. Interview Deep Dive Topics

Be ready to explain:

- Why Redis is single-threaded
 - Why LRU needs DLL + HashMap
 - How consistent hashing works
 - WAL vs Snapshot
 - Gossip protocol
 - Memtable vs SSTable
 - Compaction
 - Replica synchronization
 - How Redis Cluster works
 - How Cassandra handles partitions
 - CAP theorem tradeoffs
-

20. Final Architecture Summary

```
Client SDK
|
Gateway / LB
|
Consistent Hashing
|
Distributed Cache Nodes
|---- In-memory store
|---- TTL manager
|---- Eviction
|---- WAL
|---- Replication
|
Persistence Layer
|
Coordinator (ZooKeeper/etcd)
```